Group 9: Six Sigma

Egle Caudullo, Andrea Bianco, Federico Baio, Alessio Pistone, Alice Bianchi, Festus Udealor

Contents

The libraries **3**

The datasets **4**

The clients dataset **4**

Importing the data . . . 4

The format transformation . . . 4

NA's treatment . . . 4

Connections and correlations between the variables . . . 5

Numeric - numeric . . . 5

Categorical - categorical . . . 5

PhoneService and MultipleLines . . . 7

Internet - related variables . . . 7

OnlineBackup vs OnlineSecurity. . . . 8

Creation of new variables . . . 9

The phoneline variable . . . 9

The internet variable . . . 9

The streaming variable . . . 10

The TotalCharges variable . . . 10

The complaints dataset **12**

Sentiment analysis . . . 12

Merging the datasets . . . 14

Topic modeling . . . 16

Analysis on the full merged dataset **18**

The significance of the variables . . . 18

Models for prediction **20**

Logistic model 20

Lasso model 20

Boosting with tuning 21

Random forest 21

Results 22

The libraries

Here, a list of the libraries available in R that we have used in our analysis:

```
library(tidyverse)
library(rstatix)
library(cluster)
library(gridExtra)
library(grid)
library(caret)
library(knitr)
library(DescTools)
library(rmarkdown)
library(scales)
library(gbm)
library(glmnet)
library(randomForest)
library(tree)
library(tidytext)
library(tm)
library(RColorBrewer)
library(wordcloud)
library(xgboost)
library(readr)
library(dplyr)
library(ggplot2)
library(tidyr)
library(e1071)
library(kableExtra)
library(colorspace)
library(lsa)
```

Note: most of the results we obtained are shown through tables, created solely for aesthetic purposes. However, the values are the ones we obtained by running the code in R.

The datasets

The datasets we worked on are the `ABCTelco_clients` dataset and the `ABCTelco_complaints` dataset. Both of them contains information on the customers of a telecom company. Our goal is to merge these two datasets and combine the information that they provide by defining models that are able to predict whether the customer has left the company.
We begin with data loading and preprocessing, then move to variable analysis and modeling.

The clients dataset

Importing the data

We loaded the dataset `ABCTelco_clients.csv`. The dataset contains informations on 7043 clients of a telecom company.

```
clients <- read.csv("ABCTelco_clients.csv")
```

The format transformation

We observed that many variables in our dataset were in `character` format, while in reality, they are categorical variables.
For this reason, we created the `convert_chr_to_factor` function, which allowed us to quickly change their format and transform all the `character` variables into `factor` variables.
The `SeniorCitizen` variable, initially `numeric`, was also transformed into a `factor`.
Finally, we transformed the response variable `Churn` (which initially contained "*Yes*" and "*No*") into a `numeric` variable, where 1 represents "*Yes*" and 0 represents "*No*".

```
convert_chr_to_factor <- function(data) {
  data %>%
    mutate(across(where(is.character), as.factor))
}

clients <- convert_chr_to_factor(clients)
clients$SeniorCitizen<-as.factor(clients$SeniorCitizen)

clients$Churn <- ifelse(clients$Churn == "Yes", 1, 0)
```

NA's treatment

We then checked for the presence of missing values within our dataset and discovered the presence of 11 NAs in the `TotalCharges` variable. The summary of `df$TotalCharges` is available in the `Table 1`.

```
summary(clients$TotalCharges)
```

Table 1: summary of TotalCharges)

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
18.8	401.4	1397.5	2283.3	3794.7	8684.8	11

To address this issue, we decided to remove all the rows corresponding to these IDs. This decision was straightforward, as the large size of our dataset meant that losing 11 individuals would not result in a significant loss of information.

```
clients <- clients %>% filter(!is.na(TotalCharges))
```

Connections and correlations between the variables

In this section, we analyzed the correlation between the independent variables in our dataset. There are several types of variables: binary, categorical with multiple levels, and numeric. For each possible pair of variables, a subsection was created.

Numeric - numeric

Let's start analyzing the correlation between the numerical variables in our dataset. We have extracted the numerical variables with the function `sapply`, creating a reduced dataset called `num_var`, containing only `tenure`, `MonthlyCharges`, `TotalCharges` and `Churn`.
In `cor_matrix` the correlation between the variables is computed.

```
num_var <- clients[, sapply(clients, is.numeric)]
cor_matrix <- round(cor(num_var, use = "complete.obs", method = "pearson"), 3)
```

In the following table the correlation matrix is shown:

Table 2: Correlation Matrix between numerical variables

	tenure	MonthlyCharges	TotalCharges	Churn
tenure	1.000	0.247	0.826	-0.354
MonthlyCharges	0.247	1.000	0.651	0.193
TotalCharges	0.826	0.651	1.000	-0.199
Churn	-0.354	0.193	-0.199	1.000

As shown in `Table 3`, both `MonthlyCharges` and `tenure` are highly correlated with the variable `TotalCharges`. We will analyze this correlation in more detail later.

Categorical - categorical

In this section, we analyzed the association between pairs of categorical variables using the function `VCramers`, which we developed. The function takes a set of variables (which do not necessarily have to be categorical) as input and returns a value that measures the association between the input pairs of categorical variables. The value of this statistic ranges from 0 to 1, where a value close to 0 indicates low association between the two variables, and a value near 1 indicates high association.

```r
VCramers <- function(data) {
  cat_data <- data[, sapply(data, function(x) is.factor(x))]
  n <- ncol(cat_data)
  mat <- matrix(NA, nrow = n, ncol = n)
  colnames(mat) <- rownames(mat) <- colnames(cat_data)
  for (i in 1:n) {
    for (j in 1:n) {
      if (i == j) {
        mat[i, j] <- 1
      } else {
        table <- table(cat_data[[i]], cat_data[[j]])
        mat[i, j] <- CramerV(table)
      }
    }
  }
  return(as.data.frame(mat))
}
mat_Cramer <- VCramers(clients)
mat_Cramer <- round(mat_Cramer, 3)
```

Due to the high number of categorical variables in our dataset, we decided to display only those pairs with a statistic value greater than 0.7.

In the following table the pairs of variables with a `association index` greater than 0.7 are displayed.

```r
threshold <- 0.7
mat_Cramer[lower.tri(mat_Cramer, diag = TRUE)] <- NA
corr_var <- which(abs(mat_Cramer) > threshold, arr.ind = TRUE)

corr_pairs <- data.frame(
  var1 = rownames(mat_Cramer)[corr_var[, 1]],
  var2 = colnames(mat_Cramer)[corr_var[, 2]],
  association = mat_Cramer[corr_var]
)

corr_pairs <- corr_pairs[order(-corr_pairs$association), ]
```

Table 3: Association Matrix

	var1	var2	association
1	PhoneService	MultipleLines	1.000
22	StreamingTV	StreamingMovies	0.771
20	DeviceProtection	StreamingMovies	0.736
15	DeviceProtection	StreamingTV	0.734
9	OnlineSecurity	TechSupport	0.733
11	DeviceProtection	TechSupport	0.726
2	InternetService	OnlineSecurity	0.724
8	InternetService	TechSupport	0.723
10	OnlineBackup	TechSupport	0.720
7	OnlineBackup	DeviceProtection	0.719
4	OnlineSecurity	OnlineBackup	0.718
6	OnlineSecurity	DeviceProtection	0.717
12	InternetService	StreamingTV	0.717

	var1	var2	association
16	TechSupport	StreamingTV	0.716
17	InternetService	StreamingMovies	0.716
21	TechSupport	StreamingMovies	0.716
14	OnlineBackup	StreamingTV	0.715
19	OnlineBackup	StreamingMovies	0.714
13	OnlineSecurity	StreamingTV	0.708
18	OnlineSecurity	StreamingMovies	0.708
3	InternetService	OnlineBackup	0.707
5	InternetService	DeviceProtection	0.707

PhoneService and MultipleLines

As seen in `Table 3`, the pair of variables PhoneService and MultipleLines shows a perfect correlation. Upon further analysis, we realized that the first variable indicates whether the customer has a phone line, while the second tells us whether they have 0, 1, or 2 phone lines. Naturally, all customers who have "No" for the variable PhoneService will have the value "0" for the variable MultipleLines.

```
table(clients$PhoneService)
```

Table 4: Phone services table

No	Yes
680	6352

```
table(clients$MultipleLines)
```

Table 5: Multiple lines table

No	No phone services	Yes
3385	680	2967

```
clients %>%
  filter(PhoneService == "No" &
      MultipleLines == "No phone service") %>% nrow()

> [1] 680
```

In fact, we can see that there are 680 customers who have "*No*" for the `PhoneService` variable, "*No phone service*" for the `MultipleLines` variable. Therefore, the presence of the `PhoneService` variable is redundant, and we can remove it.

Internet - related variables

During our analysis, we also noticed that for the variables `InternetService`, `OnlineSecurity`, `OnlineBackup`, `DeviceProtection`, `TechSupport`, `StreamingTV`, and `StreamingMovies`, there were consistently 1520 customers who selected "*No*" or "*No internet service*".

```r
summary(clients$InternetService)
summary(clients$OnlineSecurity)
summary(clients$OnlineBackup)
summary(clients$DeviceProtection)
summary(clients$TechSupport)
summary(clients$StreamingTV)
summary(clients$StreamingMovies)
```

Table 6: Summary of InternetService

Servizio	DSL	FiberOptic	No
InternetService	2416	3096	1520

Table 7: Summaries of the internet-related variables

Variable	No	No internet service	Yes
OnlineSecurity	3497	1520	2015
OnlineBackup	3087	1520	2425
DeviceProtection	3094	1520	2418
TechSupport	3472	1520	2040
StreamingTV	2809	1520	2703
StreamingMovies	2781	1520	2731

We then wondered if these were always the same 1520 customers selecting these values, and we found that they were. This characteristic of these variables makes them highly correlated with each other

```r
clients %>%
  filter(
    InternetService == "No" &
      OnlineSecurity == "No internet service" &
      OnlineBackup == "No internet service" &
      DeviceProtection == "No internet service" &
      TechSupport == "No internet service" &
      StreamingTV == "No internet service" &
      StreamingMovies == "No internet service"
  ) %>%
  nrow()

> [1] 1520
```

OnlineBackup vs OnlineSecurity.

We now analyze the relationship between `OnlineBackup` and `OnlineSecurity`. These two variables appear to be highly correlated, but as we can see, this high correlation is mostly due to the presence of those 1520 customers without internet service.

```r
freq_backup_security <- round(prop.table(table(clients$OnlineBackup,
                                               clients$OnlineSecurity), 1), 2)
OnlineBK_NIS <- clients[clients$OnlineBackup != "No internet service", ]
```

```r
OnlineSC_NIS <- clients[clients$OnlineSecurity != "No internet service", ]

table(OnlineBK_NIS$OnlineBackup, OnlineSC_NIS$OnlineSecurity)
```

Table 8: Backup vs Security

	No	No internet service	Yes
No	0.71	0	0.29
No internet service	0.00	1	0.00
Yes	0.54	0	0.46

`Table 8` shows that of the people with "*No*" in `OnlineSecurity` the 71% have "*No*" in `OnlineBackup`, while only 29% have "*Yes*" in `OnlineBackup`.
Of course, no one with "*No*" in `InternetService` has either of the two values for `OnlineBackup` or `OnlineSecurity`. SPIEGARE COMMENTO (???)

Creation of new variables

The phoneline variable

To improve the interpretation of the variable `MultipleLines`, we decided to create a new categorical variable called `phonelines`, which takes the value 0 if the customer has no phone lines, 1 if they have one, and 2+ if they have multiple phone lines.

```r
clients$phonelines<- factor(ifelse(clients$MultipleLines == "No phone service",0,
                                   ifelse(clients$MultipleLines == "No",1,2)))
```

After creating this variable, we will use it in the analysis, as it provides clearer insight.

The internet variable

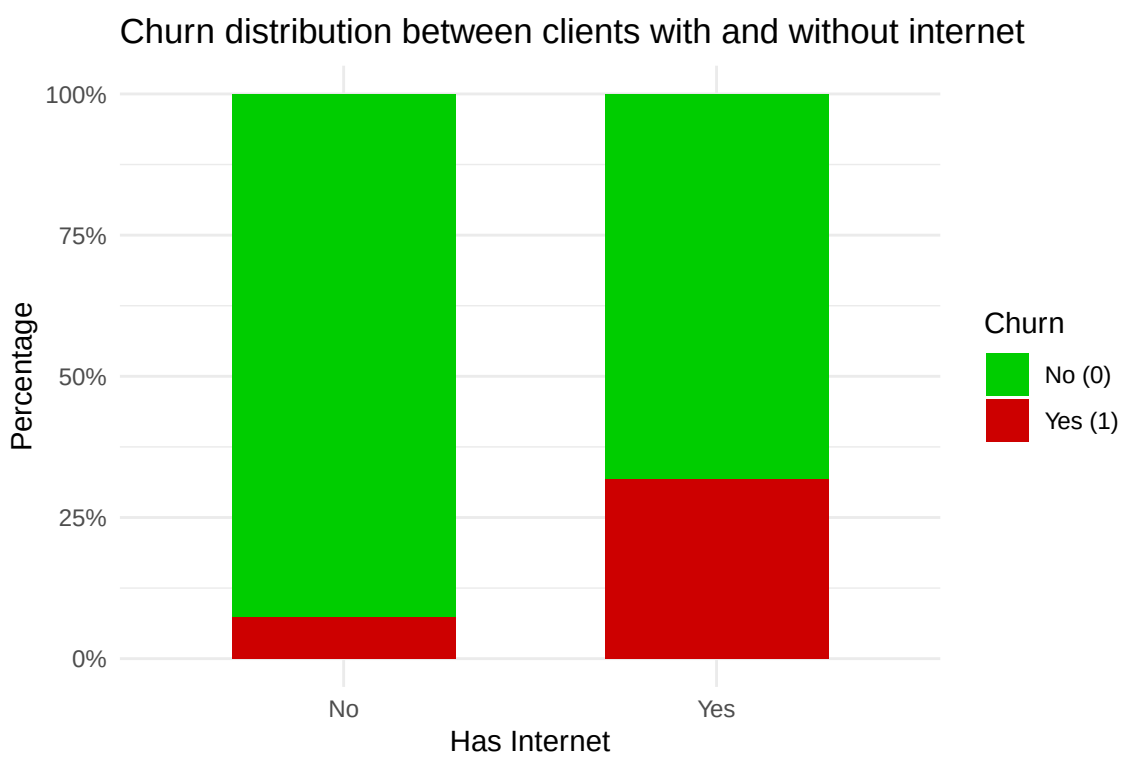
Next, we created the `internet` variable, a binary variable that takes the value "*No*" if the variable `InternetService` equals "*No*", and "*Yes*" in the other two cases (`DSL` and `FiberOptic`).

```r
clients$internet<-factor(ifelse(clients$InternetService == "No","No","Yes"))
```

We then analyzed the relationship between the newly created variable `internet` and the response variable `Churn`.
To do so, we created the frequency table `freq_internet_churn`, which is provided below.
The following matrix suggests that customers without internet access rarely churn:

	0	1
No	0.93	0.07
Yes	0.68	0.32

The relative frequencies are computed on the presence or absence of the internet service. From here, one can see that only 7% of the clients without internet service churned.



The streaming variable

Due to the high correlation between the variables `StreamingTV` and `StreamingMovies`, we decided to create a new variable called `streaming`, which takes the value 0 if the customer has no streaming service, 1 if they have only one of the two, and 2 if they have both.
The code used to compute the new variable is the following:

```
clients$streaming <- (ifelse(clients$StreamingTV =="No" &
                           clients$StreamingMovies =="No", 0,
                        ifelse(clients$StreamingTV == "Yes" &
                                 clients$StreamingMovies == "Yes", 2,
                              ifelse(clients$StreamingTV == "No" &
                                       clients$StreamingMovies == "Yes",1,
                                     ifelse (clients$StreamingTV == "Yes" &
                                               clients$StreamingMovies == "No",1,0)))))
```

The TotalCharges variable

Eventually, for informational purposes, we discovered that the `TotalCharges` variable is simply the product of `MonthlyCharges` and `tenure`. We confirmed this by creating a new variable called `new_TotalCharges`.

```
clients$new_TotalCharges <- round(clients$MonthlyCharges * clients$tenure, 2)
```

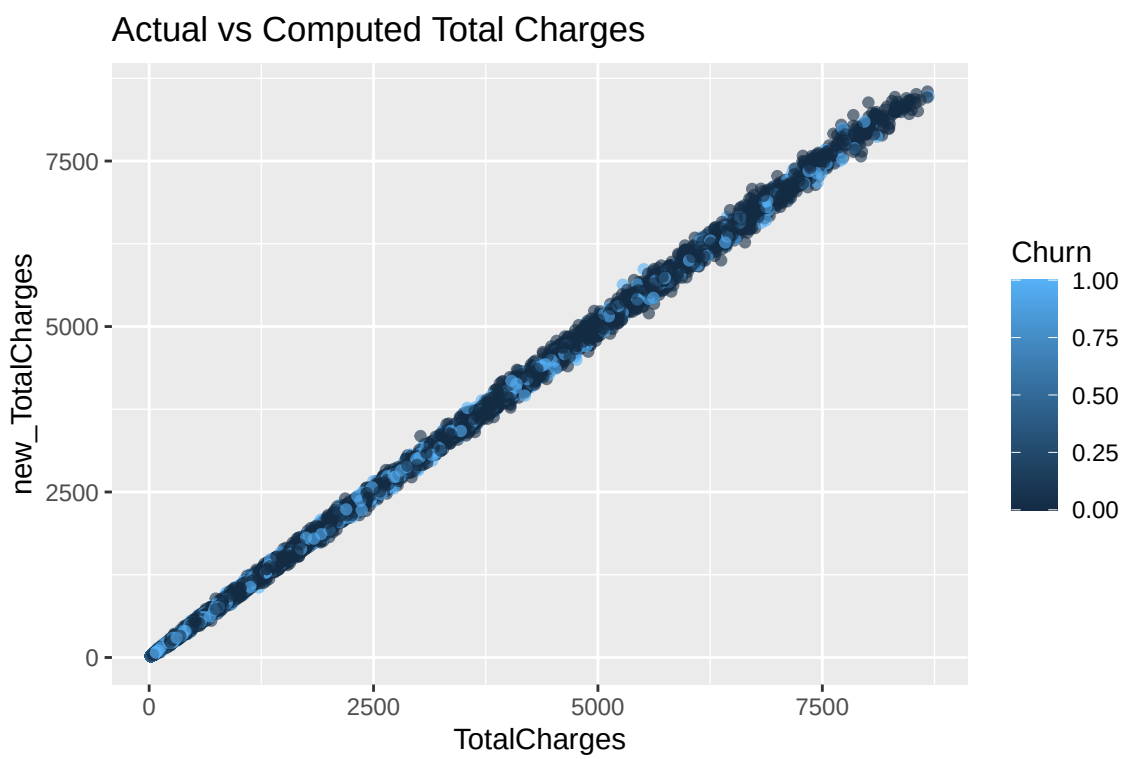
We compared the summary of `new_TotalCharges` and `TotalCharges`:

```r
summary(clients$TotalCharges)
summary(clients$new_TotalCharges)
```

Table 10: Comparison between summaries

Statistic	TotalCharges	New_TotalCharges
Min.	18.8	18.8
1st Qu.	401.4	397.8
Median	1397.5	1394.6
Mean	2283.3	2283.1
3rd Qu.	3794.7	3791.2
Max.	8684.8	8550.0

Then, by plotting this new variable against the existing `TotalCharges` variable, we observed that the computed `new_TotalCharges` shows a perfect correlation with the original variable, confirming that `TotalCharges` and `MonthlyCharges` and `tenure` provide the same information.



The complaints dataset

The dataset provides information on the complaints the customers wrote. The dataset contains 1605 observation and three variables: `customerID`, `complaint` and `complaint_number`.

Sentiment analysis

We performed a sentiment analysis in order to detect the negative sentiment in a customer complaint. We loaded the data:

```
complaints <- read.csv("ABCTelco_complaints.csv", stringsAsFactors = FALSE)
```

We changed the name of the variable `complaint` into `text`, removed the potential NA's and created a dataframe called `complaints_df`, containing only the variables `lines` and `text`.

```
texts <- complaints$complaint
texts <- texts[!is.na(texts) & texts != ""]
complaints_df <- tibble(line = 1:length(texts), text = texts)
```

Here, the initial part of the dataframe is displayed:

```
head(complaints_df)
```

line	text
1	The streaming TV service frequently buffers or crashes, making it impossible for me to watch anything without interruptions. This has been ongoing despite my high monthly charges, and I am very frustrated with the lack of reliability.
2	The DSL internet service has been inconsistent lately, with frequent slowdowns during peak hours. This is making it difficult for my family to complete their tasks online.
3	The streaming TV service often buffers excessively, even though I have a stable internet connection. This ruins the experience for my family.
4	Despite having tech support included, it takes a long time to get in touch with a representative when I call for assistance.
5	The device protection plan I pay for did not cover a recent issue with my router, and I had to pay out of pocket for repairs.
6	I requested a paper bill for my records, but the bills are often delayed, making it hard to keep track of my payments.

We then added to the dataframe the variable costumerID.
(Note: since the order of complaints is unchanged from `complaints` to `complaints_df`, each complaint can still be correctly matched to its `customerID` by simply adding the variable.)

```
comp_cust <- complaints_df %>%
  mutate(customerID = complaints$customerID)
```

We then tokenized the text into words, meaning that each complaint was divided into single words.
In the table below the first rows of `complaints_words` are reported:

```r
complaints_words <- comp_cust %>%
  unnest_tokens(word, text)
head(complaints_words)
```

line	customerID	word
1	9565-DJPIB	the
1	9565-DJPIB	streaming
1	9565-DJPIB	tv
1	9565-DJPIB	service
1	9565-DJPIB	frequently
1	9565-DJPIB	buffers

We proceeded with the removal of the *stop words*: common words but useless for the sentimental analysis, just as adverbs or conjunctions.
We loaded the `stop_words` from the package `tidytext` and with an `anti_join` we removed such words from the dataset.

```r
data("stop_words")
complaints_words_clean <- complaints_words %>%
  anti_join(stop_words, by = "word")
```

From the `tidytext` package we have loaded the `bing` lexicon, that contains thousands of words, each labeled with a sentiment: either *positive* or *negative*

```r
bing_sentiments <- get_sentiments("bing")
head(bing_sentiments)
```

word	sentiment
2-faces	negative
abnormal	negative
abolish	negative
abominable	negative
abominably	negative
abominate	negative

We joined our words from `complaints_words_clean` with the sentiment labels from `bing_sentiments`. In the table below the initial part of `complaints_sentiment` is displayed.

```r
complaints_sentiment <- complaints_words_clean %>%
  inner_join(bing_sentiments, by = "word")
head(complaints_sentiment)
```

line	customerID	word	sentiment
1	9565-DJPIB	crashes	negative
1	9565-DJPIB	impossible	negative
1	9565-DJPIB	interruptions	negative
1	9565-DJPIB	frustrated	negative

line	customerID	word	sentiment
1	9565-DJPIB	lack	negative
2	4342-HFXWS	inconsistent	negative

We associated to each complaint a numerical measure (the `sentiment_score`) based on the difference between the number of positive and negative words it contains.

```
complaint_scores <- complaints_sentiment %>%
  count(line, sentiment) %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>%
  mutate(sentiment_score = positive - negative)
```

we then merged `complaint_score` with `complaint_df`, in order to associate the computed scores to original complaints.

```
final_sentiment <- complaints_df %>%
  left_join(complaint_scores, by = "line") %>%
  mutate(sentiment_score = ifelse(is.na(sentiment_score), 0, sentiment_score))
```

Merging the datasets

In this section we merged the `ABCTelco_clients` dataset and the `final_sentiment` dataframe.
All the steps are described below.

First of all, in the original dataset `complaints`, we renamed the variable `complaint` to `text`, so that it matched the variable name in the `final_sentimen` data frame.

```
complaints1 <- complaints %>%
  rename(text = complaint) %>%
  select(-complaint_number)
head(complaints1)
```

We then added the `customerID` variable to the `final_sentimen` data frame performing a `left_join` and using as *key* the variable `text`, common to both the dataframes.

```
sentiment_id <- final_sentiment %>%
  left_join(complaints1, by = "text")
```

We kept only the `customerID` and `sentiment_score` variables:

```
sentiment_id1 <- sentiment_id %>%
  select(c(customerID, sentiment_score))
head(sentiment_id1)
```

customerID	sentiment_score
9565-DJPIB	-5
4342-HFXWS	-2
4342-HFXWS	-1

customerID	sentiment_score
4342-HFXWS	1
4342-HFXWS	0
4342-HFXWS	-2

To merge `sentiment_id1` with the `clients` dataset, the variable `customerID` must serve as a *key*. To achieve this, we performed a `group_by` operation to group all the data based on the values of `customerID`. We then computed two variables: `score`, the sum of the scores associated with each individual, and `n_complaints`, which indicates how many complaints a person has written.
Clearly, the more negative the score, the more unsatisfied the customer is; conversely, the more positive the score, the more satisfied the customer is.
Customers with a score equal to zero are considered neutral.

```
sentiment_group <- sentiment_id1 %>%
  group_by(customerID) %>%
  summarise(score = sum(sentiment_score), n_complaints = n())
head(sentiment_group)
```

customerID	score	n_complaints
0003-MKNFE	-2	3
0011-IGKFF	-3	5
0014-BMAQU	-4	3
0020-JDNXP	-2	1
0022-TCJCI	-3	4
0023-HGHWL	-6	5

creation of the variable weighted_score, which is a score built as a weighted mean in which each sentiment score is weighted by the number of tickets sent.

```
sentiment_group2 <- sentiment_group %>%
  mutate(weighted_score = (score*n_complaints)/sum(n_complaints)*100)
```

Finally, we merged the `clients` and the `sentiment_group2` datasets, using the variable `customerID` as a *key*. Given that not all the customers present in the `clients` dataset have left a complaint (the `clients` dataset contains 7043 observations while `sentiment_group2` only 704), we had to specify to put 0 instead of NA in the `score`, `n_complaints` and `weighted_score` variables.

```
df <- clients %>%
  full_join(sentiment_group2, by = "customerID") %>%
  mutate(score = replace_na(score, 0),
         n_complaints = replace_na(n_complaints, 0),
         weighted_score = replace_na(weighted_score, 0))
```

We wrote a new csv file, containing the merged dataset:

```
write_csv(df, "clientcomplain.csv")
```

Topic modeling

For information purposes, we computed the topic analysis.

Starting from the `complaint` variable of the `complaints` dataset, we prepared a, `corpus`, a list of documents, by transforming all the characters to lowercase, removing punctuation and stopwords and performing the stemming.

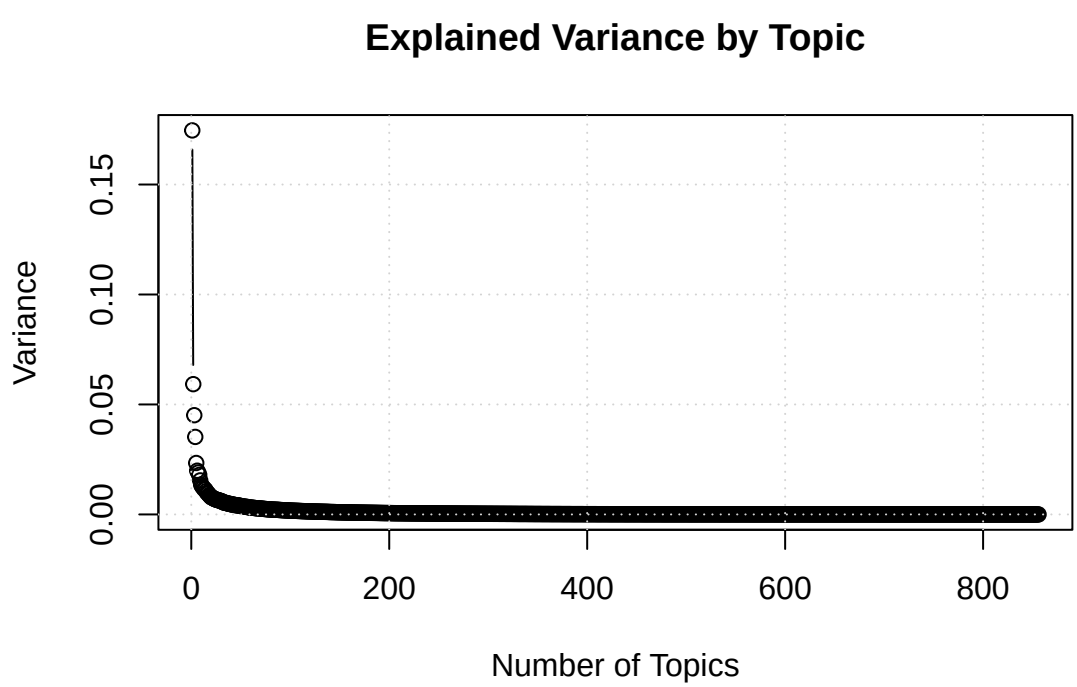
```
texts <- complaints$complaint
corpus <- Corpus(VectorSource(texts))
corpus <- tm_map(corpus, tolower)
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, function(x) removeWords(x, stopwords("english")))
corpus <- tm_map(corpus, stemDocument, language = "english")
```

We then proceeded with the latent semantic analysis.
The object `td.mat` is a matrix counting how many times the terms are used in each document.

```
td.mat <- as.matrix(TermDocumentMatrix(corpus))
td.mat<-t(td.mat)
out=svd(td.mat)
```

In the graph below we can analyze the behaviour of the explained variance as the number of topics increases:

```
plot(out$d^2 / sum(out$d^2), type = "b",
     main = "Explained Variance by Topic",
     ylab = "Variance", xlab = "Number of Topics");grid()
```



More specifically, the percentage of explained variance by the first three components is 27.9%:

```
perc_var_3 <- sum(out$d[1:3]^2) / sum(out$d^2) * 100
paste0(round(perc_var_3, 2), "%") # "27.9%"
```

Let's see how many components are needed to explain at least the 70% of the variability:

```
var_of_compon <- out$d^2 / sum(out$d^2)
cum_var <- cumsum(var_of_compon)
n_components <- which(cum_var >= 0.7)[1]
n_components #54
```

Indeed, the percentage of variance explained by first 54 components is:

```
perc_var_54 <- sum(out$d[1:54]^2) / sum(out$d^2) * 100
paste0(round(perc_var_54, 2), "%") # 70.24%
```

We obtained that 54 topics explain the 70% of the variability.
There appears to be a substantial number of topicsaccounting for a significant portion of dataset's variability.

Analysis on the full merged dataset

We loaded the merged dataset `clientcomplain` and considered the variable `customerID` as a *key* that identifies the customers.
We removed the `customerID` and the `new_TotalCharges` variables and coverted the remaining variables into `factor` variables.

```
df <- read_csv("clientcomplain.csv")
rownames(df) <- df$customerID
df$customerID <- NULL
df$new_TotalCharges <- NULL

convert_chr_to_factor <- function(data) {
  data %>%
    mutate(across(where(is.character), as.factor))
}
df <- convert_chr_to_factor(df)
df$SeniorCitizen<-as.factor(df$SeniorCitizen)
df$Churn<-as.factor(df$Churn)
df$streaming<-as.factor(df$streaming)
df$phonelines<-as.factor(df$phonelines)
str(df)
```

The significance of the variables

We performed a logistic regression including all the variables of our dataset in order to identify the significant ones.
The results are available in `Table 17`.

```
formula0 <- as.formula("Churn ~ .")
```

```
logistic <- glm(formula0, data = df, family = "binomial")
summary(logistic)
```

Table 17: Results of the logistic regression

Term	Estimate	Std..Error	z.value	p.value
(Intercept)	1.20270	0.83960	1.433	0.152
genderMale	-0.01085	0.06650	-0.163	0.87
SeniorCitizen1	0.19760	0.08680	2.277	0.0228
PartnerYes	0.00406	0.07960	0.051	0.959
DependentsYes	-0.15120	0.09190	-1.645	0.1
tenure	-0.05813	0.00639	-9.101	<2e-16
PhoneServiceYes	0.29330	0.66840	0.439	0.661
MultipleLinesNophoneservice	NA	NA	NA	NA
MultipleLinesYes	0.47140	0.18210	2.588	0.0097
InternetServiceFiberoptic	1.94670	0.82240	2.367	0.0179
InternetServiceNo	-1.98790	0.83170	-2.390	0.0168
OnlineSecurityNointernetservice	NA	NA	NA	NA
OnlineSecurityYes	-0.14850	0.18380	-0.808	0.419
OnlineBackupNointernetservice	NA	NA	NA	NA

Term	Estimate	Std..Error	z.value	p.value
OnlineBackupYes	0.05130	0.18060	0.284	0.776
DeviceProtectionNointernetservice	NA	NA	NA	NA
DeviceProtectionYes	0.17870	0.18150	0.985	0.325
TechSupportNointernetservice	NA	NA	NA	NA
TechSupportYes	-0.15690	0.18580	-0.845	0.398
StreamingTVNointernetservice	NA	NA	NA	NA
StreamingTVYes	0.66540	0.33670	1.976	0.0481
StreamingMoviesNointernetservice	NA	NA	NA	NA
StreamingMoviesYes	0.67210	0.33630	1.998	0.0457
ContractOneyear	-0.63710	0.11050	-5.763	8.25e-09
ContractTwoyear	-1.36400	0.18000	-7.576	3.56e-14
PaperlessBillingYes	0.37290	0.07660	4.871	1.11e-06
PaymentMethodCreditcard(automatic)	-0.09967	0.11680	-0.853	0.393
PaymentMethodElectroniccheck	0.29080	0.09690	3.000	0.0027
PaymentMethodMailedcheck	-0.08809	0.11820	-0.745	0.456
MonthlyCharges	-0.04759	0.03270	-1.455	0.146
TotalCharges	0.00030	0.00007	4.156	3.24e-05
phonelines1	NA	NA	NA	NA
phonelines2	NA	NA	NA	NA
internetYes	NA	NA	NA	NA
streaming1	-0.11350	0.07960	-1.427	0.154
streaming2	NA	NA	NA	NA
score	-0.50470	0.08460	-5.965	2.45e-09
n_complaints	0.39470	0.06480	6.091	1.12e-09
weighted_score	1.53910	0.40200	3.828	0.000129

Models for prediction

In this section we tried to test our dataset on different models (here we report only the best ones computationally and with respect to the misclassification error level) to see which model could better predict the Churn rate.
On all 4 models shown here we performed a 5 fold CV, to make sure to have a concrete and certain prediction on all the observations of the dataset.
We set a seed (`123`) to allow reproducibility.

```
set.seed(123)
k=5
n<-nrow(df)
folds<-sample(1:k,n,T)
err.cv.logistic<-NULL
err.cv.lasso<-NULL
err.cv.boosting<-NULL
err.cv.rf<-NULL
```

Logistic model

```
for (i in 1:k){
  x.train<-df[folds!=i,]
  x.test<-df[folds==i,]
  out.log<-glm(Churn~.,data=x.train,family="binomial")
  phat.log<-predict(out.log,newdata=x.test,type="response")
  pred.log<-ifelse(phat.log>0.5,1,0)
  err.cv.logistic[i]<-mean(pred.log!=x.test$Churn)
}
print("LOGISTIC")
mean(err.cv.logistic)
```

Lasso model

```
for (i in 1:k){
  x.train<-df[folds!=i,]
  x.test<-df[folds==i,]
  x.train.mat<-model.matrix(Churn~.,data=x.train)[,-1]
  y.train<-x.train$Churn
  cv.lasso<-cv.glmnet(x.train.mat,y.train,family ="binomial",alpha = 1)
  x.test.mat<-model.matrix(Churn~.,data=x.test)[,-1]
  phat.lasso<-predict(cv.lasso,newx=x.test.mat,s="lambda.min",type="response")
  pred.lasso<-ifelse(phat.lasso>0.5,1,0)
  err.cv.lasso[i]<-mean(pred.lasso!=x.test$Churn)
}
print("LASSO")
mean(err.cv.lasso)
```

Boosting with tuning

```
numb<-c(300,400,500,600,700)
depths<-c(1, 2, 3)
shrinkages<-c(0.01, 0.05, 0.1)
grid<-expand.grid(n.trees=numb,depth=depths,shrink=shrinkages)
err.grid.selection<-matrix(NA,nrow=k,ncol=nrow(grid))
df$Churn<-as.numeric(df$Churn)-1
for (i in 1:k){
  x.train<-df[folds!=i,]
  x.test<-df[folds==i,]
  folds2<-sample(1:k,nrow(x.train),replace=T)
  for (s in 1:k){
    cv.train<-x.train[folds2!=s,]
    cv.test<-x.train[folds2==s,]
    for (j in 1:nrow(grid)){
      boost.out<-gbm(Churn~.,data=cv.train,distribution = "bernoulli",
                     n.trees=grid$n.trees[j],
                     interaction.depth=grid$depth[j],
                     bag.fraction=1,shrinkage=grid$shrink[j])
      phat.grid<-predict(boost.out,newdata=cv.test,n.tree=grid$n.trees[j],
                         type="response")
      pred.grid<-ifelse(phat.grid>0.5,1,0)
      err.grid.selection[s,j]<-mean(pred.grid!=cv.test$Churn)
    }
  }
  best.idx<-which.min(colMeans(err.grid.selection))
  best.params<-grid[best.idx, ]
  boost.out<-gbm(Churn~.,data=x.train,distribution = "bernoulli",
                 n.trees=best.params$n.trees,
                 interaction.depth=best.params$depth,bag.fraction = 1,
                 shrinkage=best.params$shrink)
  phat.boost<-predict(boost.out,newdata=x.test,
                      n.tree=best.params$n.trees,type="response")
  pred.boosting<-ifelse(phat.boost>0.5,1,0)
  err.cv.boosting[i]<-mean(pred.boosting!=x.test$Churn)
}
print("BOOSTING")
mean(err.cv.boosting)
```

Random forest

```
df$Churn<-as.factor(df$Churn)
for (i in 1:k){
  x.train<-df[folds!=i,]
  x.test<-df[folds==i,]
  y.train<-df[folds!=i,]$Churn
  y.test<-df[folds==i,]$Churn
  x.train$Churn<-as.factor(x.train$Churn)
  rf.clas<-randomForest(Churn~.,data=x.train,importance=TRUE, ntree=300)
  pred.rf<-predict(rf.clas,newdata=x.test,type="response")
```

```
  err.cv.rf[i]<-mean(pred.rf!=y.test)
}
print("RANDOM FOREST")
mean(err.cv.rf)
```

Results

In the table below the missclassification error of each model is printed:

Table 18: Misclassification Error for each model

Model	Misclassification_Error
Logistic	0.1857838
Lasso	0.1846337
Boosting	0.1870312
Random Forest	0.1892892

The first two models, namely logistic and lasso are clearly seen as those that return the lowest errors, with a misclassification error of 0.185 and 0.184 respectively. This small improvement is probably due to the penalty that the lasso introduces, managing to bring to zero some of the coefficients of the predictors.
The second approach we tried is that of trees, respectively with boosting and random forest. Neither of these two models can improve the misclassification error, even with (as in the case of boosting) tuning on almost all possible parameters.
Additional models (such as svm, knn, bagging, etc.) return even higher errors, not making them noteworthy either.
Although it may seem intelligent to try a stacking or assembling approach, one can quickly verify how the predictions of the models are wrong on the same observations, looking at the correlation of the prediction, which for the 4 models remains very high, thus making it computationally unconvenient compared to the (not even certain) misclassification error reduction.